

=====

=====

PROJECT CONTEXT SUMMARY

Unity-Python Reinforcement Learning for Satellite Debris Capture

=====

=====

Last Updated: November 2025

Purpose: Complete contextual reference for AI assistants and future developers

=====

=====

1. PROJECT OVERVIEW & GOALS

=====

=====

Primary Objective:

Train a reinforcement learning agent to autonomously control a satellite (CoLESLaW-01) to approach, stabilize near, and eventually capture a tumbling piece of space debris using robotic manipulator arms.

This is a solo learning project focused on:

- Understanding machine learning and AI fundamentals
- Exploring robotics control in simulated space environments
- Developing practical skills in Unity-Python integration
- Building a maintainable, extensible RL training pipeline

Project Scope:

CURRENT PHASE: "Approach and Capture" behavior

- Agent learns to approach target object
- Maintain stable hover at ~1.5 Unity unit distance
- Match angular velocity with target in a common reference frame
- Align satellite orientation (+Y axis toward target)
- Minimize fuel consumption and excessive movement
- Grasping behavior (using gripper pads)
- Debris stabilization (damping target rotation)
- Safe capture and retrieval
- Multi-debris scenarios

=====

=====

2. SIMULATION ENVIRONMENT - UNITY SCENE

=====

=====

Unity Version: 2022.3.62f2

Physics: Manual stepping (SimulationMode.Script) at 50Hz (0.02s timestep)

Scene Components:

PRIMARY AGENT: CoLESLaW-01 Satellite

-
- Custom-built satellite prefab with modular robotic architecture
 - Name origin: Likely acronym (specific meaning not documented in code)
 - Prefab reinstantiated from scratch on each episode reset (clean state)

Physical Structure:

- Central body with rigidbody physics
- 4 robotic manipulator arms arranged radially
- Each arm has multiple segments connected by actuators (hinges)
- Gripper pads at end of each arm for eventual capture

Actuator System:

- 16 actuators total (4 per arm, controlling joint angles)
- Each actuator:
 - * Range: Defined min/max angles (degrees)
 - * Torque and power specifications
 - * Controlled via 0.0-1.0 input from RL agent
- Actuators implement `IPhysicsSteppable` interface
- Updated during manual physics stepping

Thruster System:

- 24 RCS (Reaction Control System) thrusters
- Distributed around satellite body for 6-DOF control
- Each thruster:
 - * Force output configurable
 - * Controlled via 0.0-1.0 input from RL agent
 - * Fuel consumption tracked (arbitrary units)
- Enable translation and rotation in 3D space

Sensor Suite:

- LiDAR system (1 unit):
 - * Returns array of raycast hit distances
 - * Currently 256 samples (`MAX_LIDAR_SAMPLES` constant)
 - * Provides spatial awareness of target and environment
- Gripper Pads (4 units, one per arm):
 - * Detect contact with target object
 - * Provide tactile feedback for grasping
 - * Return observation data via `GetMLObservation()`
- Fuel Tank sensor:
 - * Monitors remaining fuel (kilograms)
 - * Feeds into observations for fuel-conscious behavior

Internal Tracking:

- Fuel consumption accumulated per episode
- All sensors/plants registered via `IModel` interface
- Uses reflection-based registration system (`RoboticsDataClasses.cs`)

TARGET OBJECT: Space Debris

- `GameObject` representing tumbling space debris
- Has rigidbody with physics
- Has collider for collision detection
- NOT reinstantiated on reset (unlike satellite)
 - * May be modified in future to use prefab instantiation
 - * Currently just reset to starting transform

ENVIRONMENT BOUNDS:

- Episode terminates if satellite drifts >20 Unity units from target
- Episode terminates after 25 seconds (maxEpisodeTime)
- No physical boundaries or walls in scene
- Open space simulation (no ground, no obstacles)

3. EPISODE LIFECYCLE & RANDOMIZATION

Episode Start (Reset):

Triggered by:

- Python script calling env.reset()
- Unity receives RESET opcode with random seed
- EnvironmentController.ResetEnvironment() executed

Randomization Process:

1. Satellite Prefab Instantiation:

- Old satellite GameObject destroyed with DestroyImmediate()
- Fresh instance instantiated from satelliteGameObjectPrefab
- Ensures completely clean state (no residual forces/velocities)
- All references refreshed (rigidbodies, IModel interface)
- Physics objects recollected (IPhysicsSteppable)

2. Satellite Randomization:

Position:

- Random offset from starting position
- Maximum: 1.5 Unity units (maximumPositionalOffset)
- Random direction on unit sphere

Velocity:

- Random initial linear velocity
- Maximum: 0.5 units/second (maximumStartingSpeed)
- Random direction on unit sphere

Rotation:

- Random angular offset from starting orientation
- Maximum: 20 degrees (maximumAngularOffset)
- CONSTRAINED: prevents extreme initial orientations

Angular Velocity:

- Random spin on random axis
- Maximum: 15 degrees/second (maximumAngularSpeed)

3. Target Randomization:

Position:

- NOT randomized (remains at starting position)
- May be added in future updates

Rotation:

- Random rotation around random axis
- UNCONSTRAINED: full 360-degree freedom
- Creates unpredictable tumbling motion

Angular Velocity:

- Random spin on random axis
- Maximum: 15 degrees/second (maximumAngularSpeed)
- Simulates realistic debris tumbling

4. State Tracking Reset:

- episodeDone = false
- episodeReward = 0
- elapsedTime = 0
- Fuel consumption counters reset

Episode Execution:

Each step (0.02s physics timestep):

1. Python sends action array (40 floats: 16 actuators + 24 thrusters)
2. Unity applies actions to plants via IModel interface
3. Manual physics simulation stepped (Physics.Simulate())
4. Observations collected from all sensors
5. Reward calculated based on current state
6. Done flag checked (timeout or drift)
7. Response sent back to Python (obs, reward, done)

Episode Termination:

Episode ends when:

- Time limit reached (25 seconds)
- Satellite drifts too far (>20 units from target)
- (Future: Successful capture condition)

On termination:

- episodeDone flag set to true
- Final reward tallied
- Statistics available for Python logging
- Ready for reset and new episode

=====

=====

4. OBSERVATION SPACE (42+ dimensions)

=====

=====

The agent receives the following observations each step:

Generic Scene Data:

- 1-3. Relative Position (float3):
 - Vector from satellite origin to target origin
 - In satellite's local reference frame
 - Distance and direction information combined
- 4-6. Relative Velocity (float3):
 - Satellite velocity minus target velocity
 - In satellite's local reference frame
 - Closing rate information
- 7-9. Target Direction (float3):

- Unit vector pointing toward target
- In satellite's local reference frame
- Pure directional information (normalized)

10-12. Angular Velocity - Satellite (float3):

- Satellite's rotation rate
- In satellite's local reference frame
- Indicates spinning/tumbling

13-15. Angular Velocity - Target (float3):

- Target's rotation rate
- In target's local reference frame
- Debris tumbling information

Dynamic Sensor Data:

16. Fuel Tank (1 float):

- Current fuel remaining in kilograms
- Depletes as thrusters fire
- Encourages fuel-efficient behavior

17-32. Actuator States (16 floats):

- One float per actuator
- Returns normalized joint angle or similar state
- Provides proprioceptive feedback

33-36. Gripper Pad States (4 floats):

- One float per gripper pad
- Contact detection or proximity information
- Currently not heavily utilized (grasping not implemented)

37-292. LiDAR Data (256 floats):

- Array of raycast hit distances
- Spatial map of surrounding environment
- Primarily detects target object geometry
- MAX_LIDAR_SAMPLES = 256 (configurable constant)

TOTAL OBSERVATION SIZE: 292 floats (approximately)

Note: Exact size determined at runtime via handshake

May vary if sensor configuration changes

=====

=====

5. ACTION SPACE (40 dimensions)

=====

=====

The agent outputs 40 continuous actions each step:

Actions 0-15: Actuators (16 floats)

- Control joint angles of robotic arms
- Range: 0.0 to 1.0 (mapped to actuator ranges internally)
- One action per actuator
- 4 arms × 4 actuators per arm = 16 total

Actions 16-39: Thrusters (24 floats)

- ```

```
- Control RCS thruster firing intensity
  - Range: 0.0 to 1.0 (0 = off, 1 = full thrust)
  - One action per thruster
  - 24 thrusters distributed around satellite body
  - Enable 6-DOF control (translation + rotation)

Action Processing:

- ```
-----
```
- Actions clamped to [0, 1] range in Unity
 - Applied to plants via IModel interface
 - ReinforcementLearning.ApproachAndCaptureProject.Actions struct
 - Each plant (actuator/thruster) reads its corresponding action value
 - Plants update during physics step

```
=====
```

6. REWARD FUNCTION (CURRENT IMPLEMENTATION)

```
=====
```

The reward function was removed as not to evoke bias in this task.

```
=====
```

7. TRAINING INFRASTRUCTURE

```
=====
```

Algorithm: PPO (Proximal Policy Optimization)
Framework: Stable-Baselines3
Policy: MlpPolicy (Multi-Layer Perceptron)

Current Hyperparameters:

- ```

```
- Learning Rate: 3e-4
  - N Steps: 2048 (rollout length)
  - Batch Size: 64
  - N Epochs: 10
  - Gamma: 0.99 (discount factor)
  - GAE Lambda: 0.95
  - Clip Range: 0.2
  - Total Timesteps: 1,000,000
  - Eval Frequency: 50,000 steps
  - Eval Episodes: 5

Training Results (as of last session):

- ```
-----
```
- Training runs successfully complete
 - TensorBoard integration fully functional
 - Mean episode reward: ~-287 to -236 (improving)
 - Episode length: ~1090-1220 steps
 - FPS: ~76 (on user's hardware)
 - Success criteria not yet consistently met
 - Agent learning approach behavior but not stable hover

Key Challenges:

- High-dimensional action space (40 continuous actions)
- Complex physics interactions
- Sparse reward signal (bonus only when all criteria met)
- Long episodes (25 seconds at 50Hz = 1250 steps)
- Fuel management not yet prioritized

=====

8. CODEBASE ARCHITECTURE

=====

Unity Side (C#):

EnvironmentController.cs:

- Main interface between Unity and Python
- Handles step, reset, observations, rewards, termination
- References system for satellite/target GameObjects
- Episode settings and randomization logic
- Physics control (manual stepping)

UnityMLServer.cs:

- TCP server accepting Python connections
- Opcode-based binary protocol (STEP=0, RESET=1, HANDSHAKE=100)
- Thread-safe communication (network thread + Unity main thread)
- AutoResetEvent synchronization
- Sequential connection handling (one client at a time)

RoboticsDataClasses.cs:

- Core data structures for robotic systems
- ModelElements: Profiles, segments, attachment nodes
- IModel: Automatic sensor/plant discovery and registration
- Interfaces: ISensors and IPlants for all devices
- ReinforcementLearning: Observations, Actions, Rewards structs
- Reflection-based registration system

Python Side:

environments/unity_socket_client.py:

- Low-level TCP client
- Binary protocol implementation
- Automatic reconnection with exponential backoff
- Handshake for space dimension discovery

environments/unity_env_wrapper.py:

- Gymnasium-compatible wrapper
- Standard step/reset/close interface
- Space construction from handshake
- Episode tracking

algorithms/ppo/train_ppo.py:

- Main training entry point
- Model configuration
- Rich progress bars
- TensorBoard integration

- Checkpoint management
- Best model tracking

configs/config.py:

- Centralized configuration
- Connection, Logging, PPO, TensorBoard settings
- Single source of truth

utilities/:

- logger.py: Run directories, retention, logging
- tensorboard_manager.py: Auto-launch, port management
- path_utils.py: Windows long path handling (junctions)

deploy/run_model.py:

- Model deployment for demonstration
- GUI file picker + CLI arguments
- Episode loop with statistics
- Model validation and metadata display

```
=====
=====
9. PROJECT DEVELOPMENT HISTORY
=====
=====
```

Initial State (Before Conversation):

- ```

```
- Basic Unity-Python connection established
  - Training would crash during evaluation (multi-environment bug)
  - TensorBoard had never successfully launched
  - Slow startup (~20 seconds)
  - Windows long path issues
  - Code was functional but not production-ready
  - Limited error handling and logging
  - No deployment/inference capability

Major Refactor (During Conversation):

- ```
-----
```
1. Complete Python codebase rewrite
 2. New Unity server (UnityMLServer.cs replacing EnvironmentSocketServer.cs)
 3. Sequential evaluation strategy (shared connection)
 4. Robust error handling and reconnection logic
 5. TensorBoard auto-launch and proper log structure
 6. Rich progress bars and color-coded output
 7. Comprehensive logging and session tracking
 8. Best model tracking with metadata
 9. Checkpoint management with retention policy
 10. Testing framework (unit + integration)
 11. Deployment script for trained models
 12. Extensive documentation (README, inline comments)

Current State (Post-Refactor):

- ```

```
- Training runs complete without crashes
  - TensorBoard fully functional and auto-launching
  - Professional, maintainable codebase



- Comprehensive error handling
- Easy to extend to new algorithms
- Deployment capability for model demonstration
- Well-documented for future development
- Ready for continued training and experimentation

#### Development Timeline:

- Initial Unity scene and robotics system: Months prior
- Python training pipeline (v1): Weeks prior
- Critical bug identification: Start of conversation
- Complete codebase refactor: 4 days (conversation duration)
- Deployment script addition: Most recent update

### 10. KEY PROJECT CHARACTERISTICS

#### Philosophy:

- Maintainability over cleverness
- Self-documenting code with personality
- Professional but not corporate
- Designed for learning and experimentation
- Modular, extensible architecture

#### Technical Decisions:

- Windows-only (simplicity over cross-platform)
- TCP sockets (reliable, simple, well-understood)
- Binary protocol (performance over readability)
- Manual physics stepping (deterministic, controllable)
- Sequential evaluation (robust over parallel speed)
- Flat TensorBoard structure (easy comparison)
- Junction workaround (Windows path limitations)

#### Code Quality Standards:

- PEP 8 compliance (Python)
- Type hints (pragmatic, not exhaustive)
- reST docstrings
- Professional third-person comments
- Comprehensive error handling
- Color-coded console output
- Consistent patterns across all files

#### User Experience:

- Rich progress bars during training
- Detailed console output with colors
- Automatic TensorBoard launch
- GUI file picker for deployment
- Clear error messages
- Graceful degradation
- Ctrl+C interrupt handling

```
=====
=====
11. FUTURE DIRECTIONS
=====
=====
```

## 11. FUTURE DIRECTIONS

### Immediate Next Steps:

- Continue training to achieve stable hover behavior
- Fine-tune reward function if needed
- Experiment with different hyperparameters
- Monitor training progress via TensorBoard

### Short-Term Goals:

- Achieve consistent hover at 1.0 unit distance
- Implement fuel consumption penalty in reward
- Add success rate tracking
- Optimize training performance

### Medium-Term Goals:

- Implement grasping behavior phase
- Train agent to close gripper pads on target
- Add target stabilization (damping rotation)
- Multi-phase reward structure (approach → hover → grasp)

### Long-Term Vision:

- Complete capture and retrieval
- Multiple debris objects in scene
- Different debris shapes and sizes
- Obstacle avoidance
- Fuel-optimal trajectory planning
- Curriculum learning (progressive difficulty)

### Potential Enhancements:

- Visual rendering of agent behavior
- Real-time trajectory visualization
- Comparison tool for multiple models
- Hyperparameter optimization framework
- Distributed training support
- Alternative algorithms (SAC, TD3, etc.)

```
=====
=====
12. IMPORTANT IMPLEMENTATION NOTES
=====
=====
```

## 12. IMPORTANT IMPLEMENTATION NOTES

### Critical Details That May Not Be Obvious:

#### 1. Satellite Prefab Reinstantiation:

- Satellite is destroyed and recreated each episode
  - This ensures completely clean state
  - Target is NOT reinstantiated (may change)
  - All references must be refreshed after instantiation
2. Physics Stepping:
    - Manual physics stepping at 50Hz
    - SimulationMode.Script (not FixedUpdate)
    - IPhysicsSteppable interface for custom physics objects
    - Physics.Simulate() called explicitly
  3. IModel Interface:
    - Reflection-based sensor/plant discovery
    - Automatic registration on instantiation
    - No manual wiring required
    - Extensible to new device types
  4. Action/Observation Sizes:
    - Dynamically discovered via handshake
    - Not hardcoded in Python
    - Class-level storage (shared across instances)
    - Validated on each connection
  5. Episode Randomization:
    - Satellite position AND velocity randomized
    - Satellite rotation CONSTRAINED ( $\pm 20^\circ$ )
    - Target rotation UNCONSTRAINED (full  $360^\circ$ )
    - Creates diverse initial conditions
  6. Reward Function Origin:
    - Generated by Claude Sonnet 4.5
    - Well-designed multi-component structure
    - Not user-designed (but understood and accepted)
    - May be modified as training progresses
  7. Connection Handling:
    - Sequential, not parallel
    - Single Unity instance serves one Python client
    - Evaluation shares training connection
    - Auto-reconnection on connection loss
  8. File Paths:
    - Windows path limitations handled automatically
    - Junction creation at C:/Junctions/
    - Transparent to user
    - No manual intervention required
  9. TensorBoard Structure:
    - Flattened for easy comparison
    - All runs in tensorboard/ directory
    - Per-run data in separate directories
    - Best model tracked separately
  10. Testing Philosophy:
    - Unit tests mock Unity connection
    - Integration tests require running Unity
    - Custom test runner for readability
    - pytest under the hood

```
=====
=====
13. CONTEXT FOR AI ASSISTANTS
=====
=====
```

### 13. CONTEXT FOR AI ASSISTANTS

When Working With This Project:

-----

#### Key Files to Understand:

- RoboticsDataClasses.cs: Core data structures and RL interfaces
- EnvironmentController.cs: Unity-side RL logic
- UnityMLServer.cs: Communication protocol
- unity\_env\_wrapper.py: Python-side environment interface
- train\_ppo.py: Training pipeline
- config.py: All configuration

#### Domain Knowledge Required:

- Reinforcement learning fundamentals (PPO algorithm)
- Unity physics and GameObject lifecycle
- TCP socket communication
- Stable-Baselines3 API
- Gymnasium environment interface

#### Project-Specific Terminology:

- CoLESLaW: The satellite agent
- Plants: Output devices (actuators, thrusters)
- Sensors: Input devices (LiDAR, gripper pads, fuel tank)
- IModel: Interface for sensor/plant discovery
- Episode randomization: Initial state variation
- Manual physics stepping: SimulationMode.Script

#### Common Misconceptions to Avoid:

- This is NOT multi-agent training (single satellite)
- Unity scene does NOT have multiple environments in parallel
- Evaluation does NOT require separate Unity instance
- Model files are NOT algorithm-specific (auto-detected)
- Observations are NOT image-based (numerical vectors)

#### Design Patterns to Preserve:

- Sequential connection handling (don't parallelize)
- Shared evaluation connection (don't create separate)
- Color-coded console output (maintain consistency)
- Ctrl+C interrupt handling (graceful shutdown)
- Automatic reconnection on connection loss
- Flat TensorBoard structure (easy comparison)

#### Critical Constraints:

- Windows-only (don't add cross-platform complexity)
- Python 3.12.4 (dependency compatibility)
- Unity 2022.3.62f2 (API compatibility)
- Single Unity instance (don't require multiple)
- Self-contained Python directory (no external paths)

```
=====
=====
14. CONTACT & VERSION INFORMATION
=====
=====
```

```
Project Owner: [User - St Helens, England, GB]
AI Assistant: Claude (Anthropic) - Multiple sessions
Development Period: November 2025
Current Version: 2.0 (post-refactor)
```

```
Key Conversation Milestones:

```

1. Initial debugging session (multi-environment crash)
2. Comprehensive requirements gathering (checklist)
3. Complete Python-side rewrite
4. Unity server (UnityMLServer.cs) creation
5. Testing framework implementation
6. Deployment script addition
7. This context document creation

```
Tools Used:

```

- Unity 2022.3.62f2
- Python 3.12.4
- Visual Studio Code
- Stable-Baselines3
- TensorBoard
- Rich (progress bars)
- pytest (testing)

```
External Resources:

```

- Stable-Baselines3 documentation
- Gymnasium API reference
- Unity scripting reference
- PPO algorithm paper (Schulman et al.)

```
=====
=====
END OF CONTEXT DOCUMENT
=====
=====
```

This document provides comprehensive context about the satellite debris capture reinforcement learning project. It captures details that exist in the Unity scene, developer intentions, and design decisions that may not be immediately apparent from code inspection alone.

For AI assistants working with this project: Read this entire document before making suggestions or modifications. The project has specific constraints, design patterns, and domain knowledge that should be preserved.

Last updated: November 2025

Maintained by: Project developer with AI assistance